

Full Length Article

LGS-KT: Integrating logical and grammatical skills for effective programming knowledge tracing

Xinjie Sun^{a,b,d}, Qi Liu^{c,d,*}, Kai Zhang^{b,d}, Shuanghong Shen^{b,d}, Yan Zhuang^{b,d}, Yuxiang Guo^{b,d}^a School of Computer Science, Liupanshui Normal University, Liupanshui, China^b School of Computer Science and Technology, University of Science and Technology of China, Hefei, China^c School of Artificial Intelligence and Data Science, University of Science and Technology of China, Hefei, China^d State Key Laboratory of Cognitive Intelligence, Hefei, China

ARTICLE INFO

Keywords:

Knowledge tracing

Data mining

Logical skill

Grammatical skill

Programming education

ABSTRACT

Knowledge tracing (KT) estimates students' mastery of knowledge concepts or skills by analyzing their historical interactions. Although general KT methods have effectively assessed students' knowledge states, specific measurements of students' programming skills remain insufficient. Existing studies mainly rely on exercise outcomes and do not fully utilize behavioral data during the programming process. Therefore, we integrate a *Logical and Grammar Skills Knowledge Tracing (LGS-KT)* model to enhance programming education. This model integrates static analysis and dynamic monitoring (such as CPU and memory consumption) to evaluate code elements, providing a thorough assessment of code quality. By analyzing students' multiple iterations on the same programming problem, we constructed a reweighted logical skill evolution graph to assess the development of students' logical skills. Additionally, to enhance the interactions among representations with similar grammatical skills, we developed a grammatical skills interaction graph based on the similarity of knowledge concepts. This approach significantly improves the accuracy of inferring students' programming grammatical skill states. The LGS-KT model has demonstrated superior performance in predicting student outcomes. Our research highlights the potential application of a KT model that integrates logical and grammatical skills in programming exercises. To support reproducible research, we have published the data and code at <https://github.com/xinjiesun-ustc/LGS-KT>, encouraging further innovation in this field.

1. Introduction

Knowledge tracing (KT) refers to the dynamic estimation of students' mastery of knowledge concepts (KCs) or skills (such as loops) based on their responses to problems. These estimations are then used to predict their future performance (Pandey & Karypis, 2019; Shen et al., 2024), as shown in Fig. 1. The importance of programming skills is becoming increasingly prominent, and more people are encouraged to engage in programming exercises to enhance their individual programming abilities (Lindberg, Laine, & Haaranen, 2019; Messer, Brown, Kölling, & Shi, 2024). In this process, tracking learners' programming skills becomes crucial (Ashurova & Ashurov, 2023; Huang, Brusilovsky, Guerra, Koedinger, & Schunn, 2023; Li et al., 2022).

In recent years, deep learning knowledge tracing (DLKT) models have flourished, encompassing autoregressive deep sequential KT models (Chen, Liu, Huang, Liu, & Luo, 2023; Minn, Yu, Desmarais, Zhu, & Vie, 2018; Wang et al., 2023), attention mechanisms (Ghosh,

Heffernan, & Lan, 2020; Pandey & Karypis, 2019), and graph-based KT models (Nakagawa, Iwasawa, & Matsuo, 2019; Yang et al., 2021). Recently, programming KT tasks have garnered significant attention from researchers. To enhance the ability to predict students' programming behaviors and knowledge states (Li et al., 2022; Liu, Wang, Baraniuk, & Lan, 2022), various methods have been proposed. For instance, Code-DKT (Shi, Chi, Barnes, & Price, 2022) integrates Code2Vec with the DKT model (Piech et al., 2015) to embed student code and model knowledge states; the PTDK model (Zhu et al., 2022), on the other hand, employs dual-sequence modeling techniques to accurately predict the outcomes of students' subsequent submissions.

Although DLKT methods have introduced new perspectives for handling general KT issues, the programming exercise process (PEP) significantly differs from general exercise processes. In general exercises, learners answer questions in sequence, and each question is not attempted multiple times in a row, as shown in Fig. 1(a). In contrast, PEP

* Corresponding author.

E-mail addresses: xinjiesun@mail.ustc.edu.cn (X. Sun), qiliuql@ustc.edu.cn (Q. Liu), kkzhang08@ustc.edu.cn (K. Zhang), closer@mail.ustc.edu.cn (S. Shen), zykb@mail.ustc.edu.cn (Y. Zhuang), guoyx18@mail.ustc.edu.cn (Y. Guo).<https://doi.org/10.1016/j.neunet.2025.107164>

Received 7 May 2024; Received in revised form 15 November 2024; Accepted 10 January 2025

Available online 18 January 2025

0893-6080/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

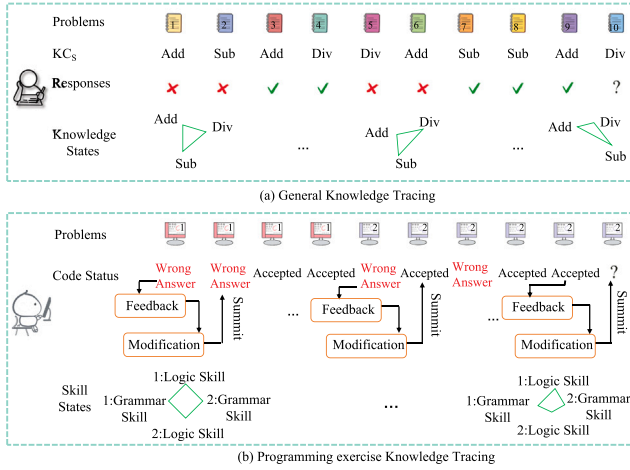


Fig. 1. A toy example demonstrating the process of KT for general KT and programming exercise KT.

allows learners to solve problems through an iterative process. In each exercise, learners can submit code, receive feedback, and repeatedly modify their solutions, as depicted in Fig. 1(b). Unlike general exercises (such as multiple-choice math questions), a learner's proficiency is not only reflected in whether they answer questions correctly but is also directly manifested in the programming skills embodied in the code they submit. This includes the relevance of grammatical skills used across different programming questions and the evolution of students' programming logical skills during the continuous code iteration process within the same question. This process generates richer behavioral data for each learner than general exercises, requiring more careful consideration when assessing programming skills proficiency.

To ensure accurate measurement of skill proficiency in programming education, we must address three major challenges. **Firstly**, it is crucial to perform detailed information extraction, grammatical analysis, and code quality assessment on students' submitted code. Although students' code harbors rich information about programming skills, existing code embedding technologies (Devlin, Chang, Lee, & Toutanova, 2018; Feng et al., 2020; Guo et al., 2020; Hinton & Salakhutdinov, 2006) primarily focus on the structure and semantics of the source code, often failing to delve into data directly related to programming skills. **Secondly**, given that the same problem in programming exercises often involves continuous code iteration aimed at enhancing programming logic, effectively assessing the evolution of students' problem-solving approaches and the growth of their programming logical skills from these continuous changes poses a significant challenge. **Lastly**, uncovering the grammatical knowledge similarities between different problems and building interactive representations of grammatical skills based on these similarities are key to enhancing the accuracy and practicality of the model. To address these issues, we need to adopt a comprehensive approach that not only considers the personalized characteristics of students' programming logical skills but also deeply understands their mastery of grammatical knowledge, to achieve a more objective and precise assessment of students' programming skill states.

To effectively address the challenges mentioned above, we propose the logical and Grammatical Skills KT (LGS-KT) model, which innovates in several key aspects aimed at enhancing the effectiveness and interpretability of KT in programming exercises. **Firstly**, to deeply mine the information contained in the code, we use static analysis combined with dynamic monitoring (such as CPU and memory consumption) to assess code quality and score different programming grammar elements within the code. **Secondly**, to evaluate the development of students' logical skills during the programming process, we perform differential

analysis on multiple submissions of the same problem and, combining code quality, grammar scoring, and source code information, we construct a reweighted logical skills evolution graph. **Lastly**, to facilitate interaction among representations with similar grammatical skills, we first establish a similarity graph of knowledge concepts and then, based on this graph, extract the grammatical knowledge relationships involved by different students to build a grammar skill interaction graph, enhancing the model's ability to infer students' programming grammatical skill states. These innovations together significantly improve the model's practicality and explanatory power.

In this study, we propose a new model for KT in programming exercises, aimed at more accurately and comprehensively assessing students' programming skills. Our main contributions can be summarized as follows:

- We propose an innovative assessment method that integrates logical and grammatical skills in programming exercises to more precisely evaluate students' proficiency in programming skills.
- By constructing a reweighted logical Skills evolution graph and a grammatical skills interaction graph, our model can meticulously track and analyze changes and developments in students' skills during the programming exercise. The introduction of these student profiles provides new perspectives and deep insights into how students gradually master complex programming tasks.
- In experiments comparing our LGS-KT model with existing KT models, our model demonstrated superior performance across multiple datasets. This result confirms the effectiveness and educational value of our model in practical applications.

These contributions not only showcase the innovation of our research but also provide powerful tools for programming education practice, helping educators better understand and support students' learning processes.

2. Related works

Accurately identifying students' programming skills during coding exercises is an extremely valuable task. Subsequently, we will explore relevant research from two perspectives: KT and modeling in computer science education.

2.1. Knowledge tracing

Recently, deep learning technologies have been extensively applied to KT tasks, which model students' historical learning patterns and predict future performance. Existing DLKT methods can be categorized into two main types based on their application scenarios: academic and programming subjects.

C1: Academic Subjects Modeling the knowledge states of students in academic subjects is one of the most important applications of KT tasks. These studies not only enhance the performance of the models but also provide us with new perspectives and tools to deeply understand students' learning processes. In recent years, the application of deep learning technologies in the field of KT has continuously driven research progress. Researchers are dedicated to improving the foundational structures of models to more accurately trace students' knowledge states. By incorporating Long Short-Term Memory (LSTM) layers (Piech et al., 2015), memory networks (Abdelrahman & Wang, 2019; Sun et al., 2021; Zhang, Shi, King, & Yeung, 2017), and attention mechanisms (Ghosh et al., 2020; Huang et al., 2024; Pandey & Karypis, 2019), these methods can better differentiate students' mastery of various KCs and improve the accuracy of the models. In addition to improvements in model structure, researchers have also begun to incorporate problem-solving features to gather more comprehensive information and more accurately understand and predict students' knowledge levels. By encoding predicted responses and actual

responses as binary bits and introducing problem IDs, these efforts enable models to better consider students' problem-solving processes and performance (Cheung & Yang, 2017; Sonkar, Waters, Lan, Grimaldi, & Baraniuk, 2020).

At the same time, incorporating educational theories into deep KT models has also become an important direction of research. Simulating student behaviors, learning gains, and forgetting processes enhances the understanding of the learning process and improves the interpretability of KT (Shen, Chen, et al., 2022). Furthermore, by accurately identifying students' target hierarchies based on their responses to problems, researchers can guide the fine-grained modeling of knowledge states under these target hierarchies (Sun et al., 2024). Inspired by graph learning techniques, researchers have begun to apply technologies such as graph neural networks to flexibly model the relationships between problems and skills, bringing new perspectives and tools to the task of KT (Nakagawa et al., 2019; Yang et al., 2021). **Our LGS-KT model implements two graph networks to map the relationships between logical skills and interactions among grammatical skills during students' programming processes.**

C2: Programming Subjects In early research in this field, scholars used exam results from previous courses or learners' log data to explore potential learning difficulties (Castro-Wunsch, Ahadi, & Petersen, 2017; Gao, Marwan, & Price, 2021), but these methods rarely focused on the source code itself. Some studies aim to accurately depict the skill levels of programming learners to assess their programming knowledge in a single exercise scenario (Kasurinen & Nikula, 2009; Wang, Sy, Liu, & Piech, 2017). To better model students' programming skills, an increasing number of researchers have started to focus on the code submitted by students. For example, Code-DKT (Shi et al., 2022) embeds students' code through Code2Vec before feeding it into a DKT model to model the states of students' programming knowledge, achieving notable results. Additionally, PST (Li et al., 2022) utilizes the knowledge embedded in students' programming processes to measure learners' programming skills, achieving a more granular assessment. OKT (Liu et al., 2022) represents the first exploration of open KT, combining the program synthesis methods of language models with student KT methods to predict students' code generation. Subsequent models like PTDK (Zhu et al., 2022) consider multiple possibilities of systematic detection of students' code and implement a dual-sequence modeling process to predict the next submission result. **Our LGS-KT model provides detailed modeling of students' submitted code during programming exercises.**

2.2. Computer science education

The application of educational data mining in the field of computer education is increasingly expanding (Choi, Lam, & Mendes, 2023; Wang et al., 2022; Zhang et al., 2023), with growing attention to research on code embedding and grammar extraction. Current studies on code embedding primarily employ two strategies: structure-based and context-based. Structure-based approaches utilize parse trees to represent code, including models such as Code2vec (Alon, Zilberstein, Levy, & Yahav, 2019), ASTNN (Zhang et al., 2019), and GraphCodeBERT (Guo et al., 2020). Early context-based studies used basic text representation models, such as RNNs (Zaremba & Sutskever, 2014) or attention-based models (Iyer, Konstas, Cheung, & Zettlemoyer, 2016). In recent years, researchers have begun to directly apply natural language processing (NLP) pre-training strategies to code embedding, such as CodeBERT (Feng et al., 2020) and GPT-C (Svyatkovskiy, Deng, Fu, & Sundaresan, 2020). These models are pre-trained in large code corpora sourced from mature software engineering projects.

On the other hand, utilizing code features for student learning analysis involves attempting to directly extract meaningful knowledge components from student code. ASTs represent student code submissions, with node types (e.g., loops, conditional statements) serving as representatives of KCs (Huang et al., 2023; Rivers, Harpstead, &

Koedinger, 2016; Shi, Schmucker, Chi, Barnes, & Price, 2023). Through advanced technologies and feature extraction techniques, educators can better understand students' learning processes, offering new perspectives for the development of personalized learning and teaching methods. **In this paper, we employ CodeBERT and GraphCodeBERT for embedding, and use ASTs to extract grammatical relationships in the code.**

3. Preliminary

In this section, we will define and describe the algorithms for KT, code grammar scoring, and code quality scoring under programming exercises that integrate logical skills and grammatical skills. Additionally, we will introduce some critical student behavior embeddings involved in our proposed LGS-KT model. Table 1 summarizes the mathematical symbols used in this paper.

3.1. Problem definition

In online judging systems, the set of students is denoted by S , the set of coding problems by CP ,¹ the set of student code submitted by C , and the set of student response outcomes by R . The set of students $S = \{s_1, s_2, \dots, s_n\}$ includes n different students. The set of coding problems $CP = \{cp_1, cp_2, \dots, cp_m\}$ contains m different coding problems. The set of student code submissions $C = \{c_1, c_2, \dots, c_k\}$ includes k different code submission fragments. Each student can have $L (L \geq 1)$ records of answers to programming problems, and the same coding problem can be submitted multiple times. The set of response outcomes, $R = \{0, 1\}$, consists of two elements used to indicate the acceptance status of a student's submitted code, where 1 signifies that the code was accepted by the system, and 0 denotes scenarios where the code was not accepted.

In the context of student programming assessments, we characterize a student's historical programming activity as a sequence of coding events within the system, denoted by $L = \{(cp_1, c_1, r_1), (cp_2, c_2, r_2), \dots, (cp_t, c_t, r_t)\}$, where cp_t represents the coding problem (or knowledge concepts) addressed by the student at time step t , c_t denotes the code submitted by the student for the respective problem, and r_t signifies the response label at time step t for the coding problem cp_t , indicating the correctness of the solution. By analyzing this sequence of practice events, which spans a length of t , we can effectively track the progression of students' mastery over programming concepts. The definitions of logical skills, syntax skills, and Knowledge Tracing (KT) in programming practice are as follows:

Logical Skills: Logical skills refer to the logical thinking and problem-solving abilities demonstrated by students in programming, including analyzing problems, designing algorithms, and debugging code. Although students' internal thought processes are not directly observable, this paper evaluates logical skills through measurable aspects that include the edit distance $dw(c_t, c_{t+1})$ between consecutive submissions for the **same problem**, code quality $Q(c_t)$, and code grammar $G(c_t)$. These metrics reflect iterative improvement, solution soundness, and adherence to programming rules. The detailed calculation process is shown in Section 4.2.

Grammatical Skills: Grammatical skills refer to a student's understanding and application of the grammar rules of a programming language. This paper quantitatively evaluates these skills by analyzing students' code submissions across **different programming problems**, specifically examining the grammar of the code $G(c_t)$ and the application and transferability of grammar in similar problems CP_t . The detailed calculation process is shown in Section 4.3.

¹ In this paper, programming problems are equivalent to knowledge concepts.

Knowledge Tracing (KT): Given a student's exercise record $L = \{(c_{p1}, c_1, r_1), (c_{p2}, c_2, r_2), \dots, (c_{p_i}, c_i, r_i)\}$, the task of programming KT is a dynamic process. A student's programming skill level changes over time based on their performance in grammar and logic. The objective of this task is to assess their mastery of programming skills based on their historical code submitted and their performance in grammar and logic, and to further predict their performance in the next programming task. For students at different stages and with varying levels of programming skills, we provide appropriate feedback and interventions. For instance, for those with stronger programming skills, we offer encouragement and rewards, as well as more challenging programming tasks.

3.2. Definition of code grammar scoring

The grammar of a programming language is crucial for ensuring the executability of code. In programming practice, focusing on grammar exercises is essential for accurately understanding and applying grammatical rules, as well as for writing clear, readable, and maintainable code (Hermans, 2020; Qureshi, Asif, & Anwar, 2023). Grammar scoring can reflect the extent to which students have mastered the grammar to some degree. Initially, we categorize the grammar categories for a specific programming language, resulting in a set of grammar elements $GE = \{ge_1, ge_2, \dots, ge_n\}$, and assign different grammar weights $G = \{wg_1, wg_2, \dots, wg_n\}$ to these elements based on their varying levels of difficulty. Then, by using ASTs to extract grammar from the code submitted by students, we provide a grammar-based score for the submitted code, defined as follows:

$$Tree_t = AST(c_t), \quad G(c_t) = \sum_{i=1}^n wg_i \cdot ge_i, \quad (1)$$

where $AST(\cdot)$ is the Abstract Syntax Tree function for the respective programming language, $Tree_t$ represents the grammar extraction of the student's code snippet, and c_t denotes the source code block submitted by the student at time t .

3.3. Definition of code quality scoring

In addition to grammatical factors, the quality of the code is a crucial factor that cannot be ignored when assessing the success of problem-solving through programming (Keuning, Jeuring, & Heeren, 2023). Reliable and efficient code is another key factor in evaluating students' programming abilities. Therefore, students' attention to code quality helps to improve their programming skills and cultivate their problem-solving logical skills. This paper adopts a combination of static evaluation Q_{sa} and dynamic evaluation Q_d to score the quality of the code submitted by students. The definition of $Q(c_t)$ is as follows:

$$\begin{aligned} Q_s &= SCA(c_t), \\ Q_d &= w_c \times C + w_m \times M, \\ Q(c_t) &= g \times Q_s + (1 - g) \times Q_d, \end{aligned} \quad (2)$$

where $SCA(\cdot)$ represents the static code analysis function, such as `pylint` in Python, using the corresponding default score initialization weights, w_c , w_m , and g respectively denote the weights for CPU consumption, memory consumption, and the hyperparameters used to balance the two evaluation methods. C indicates the CPU consumption, while M represents the amount of memory consumed.

3.4. Embeddings

Before delving into the detailed structure of the LGS-KT model, we will first briefly introduce the basic embeddings of LGS-KT. This will help to gain a deeper understanding of the internal workings of the LGS-KT model.

Table 1

Mathematical symbol and descriptions.

Symbol	Description
S, CP	The set of student and programming problem
C, R	The set of submitting code and response
Q, G	The set of quality score and grammar score
n, m	The number of student and programming problem
k	The number of submitted code
L	Students' exercise sequence
$D_{i,j}$	the edit distance between codes
h_t	Students' knowledge state at time t
$c_p, \mathbf{CP}$	The programming problem and its embedding
$c, \mathbf{C}$	The submitting code and its embedding
$r, \mathbf{R}$	Submitted code acceptance and embedding
$G(c_i), \mathbf{G}$	Grammar score and its embedding
$Q(c_i), \mathbf{Q}$	Quality score and its embedding
$S_{ij}, \mathbf{S}$	Programming problem similarity and its matrix
L_{seg}	Length of interval segment
AST	Abstract Syntax Tree

3.4.1. Code quality score, code grammar score, and response embeddings

In the construction of the LGS-KT model, we primarily embed three core components: the quality score of the code (Q), the grammatical score of the code (G), and the system's response to the student-submitted code (R). To characterize the quality of the student-submitted code, we introduce an embedding matrix $\mathbf{Q} \in \mathbb{R}^{k \times d_k}$ (where d_k represents the dimension of the embedding) to represent the quality score of the student-submitted code. Simultaneously, we use an embedding matrix $\mathbf{G} \in \mathbb{R}^{k \times d_g}$ (where d_g represents the dimension of the embedding) to represent the grammatical score of the submitted code, with k representing the total number of different codes submitted by all students. Moreover, we introduce $\mathbf{R} \in \mathbb{R}^{2 \times d_r}$ (where d_r represents the dimension of the embedding, and 2 indicates whether the student's code submission is accepted by the system, with a value of 1 indicating acceptance and other values representing other responses from the online judging system) to describe the student's response situation.

3.4.2. Programming problem and submitted code representation

In the LGS-KT model, the representation of programming problems and submitted code is particularly important. To deeply understand the specific grammar, semantics, and structural rules in programming languages, we use the pretrained model GraphCodeBERT (Guo et al., 2020) to represent the source code submitted by students $\mathbf{C} \in \mathbb{R}^{K \times 768}$ (where K is the number of different source codes submitted by all students). CodeBERT (Feng et al., 2020) is used to represent the programming problems $\mathbf{CP} \in \mathbb{R}^{m \times 768}$ (where m is the total number of programming problems). The use of these two pretrained models not only enhances the performance of our model but also improves the understanding and handling of the complexity of programming languages, thereby achieving more accurate skill state assessments in the KT of student programming exercises.

4. The LGS-KT model

In this section, we will delve into the LGS-KT model proposed in this study, which consists of four main parts, as shown in Algorithm 1 and Fig. 2. First, we extract information from the source code submitted by students to obtain representations of the source code, quality scores, and grammar scores, as shown in Fig. 2(a). Next, we calculate the grammar and quality scores of the code and, based on the sequence of code submissions by students for the same problem, assess the sequential evolution of students' logical skills to construct a reweighted neural network graph of students' logical skills, as shown in Fig. 2(b). At the same time, we calculate the global similarity of programming problems (or knowledge concepts) and use this to construct a similarity graph of the problems. For each student, we filter out the structural relationships of programming problems involved in their

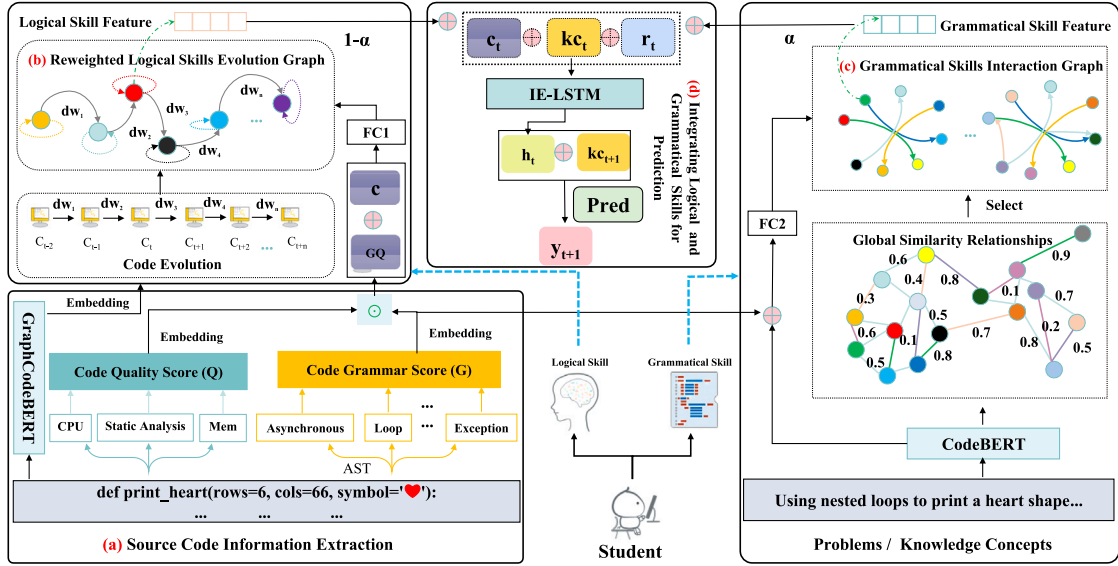


Fig. 2. The core architecture of our LGS-KT model has been presented. The diagram reveals the knowledge tracking process in programming practice. At time step t , the inputs include the submitted code c_t , the knowledge concept KC_t , the student's response r_t , and the knowledge state h_t . Additionally, we can also predict the student's performance at the next time step $t + 1$, denoted as y_{t+1} .

practice sequence to construct a neural network graph of grammatical skills interactions, as shown in Fig. 2(c). Finally, by comprehensively considering the students' logical and grammatical skills, we predict the sequence of students' programming behaviors, as shown in Fig. 2(d). In the following sections, we will provide a more detailed description and discussion of these four modules.

Algorithm 1 LGS-KT Algorithm

Require: C_t : Student's submitted code, Q_t : Code quality score, G_t : Grammar score, L : Submission sequence
Ensure: P_{t+1} : Prediction of student's behavior at time $t + 1$
1: **Input:** C_t, Q_t, G_t, L
2: **Initialization:** Extract features from C_t , compute Q_t and G_t
3: **procedure** INFORMATION EXTRACTION
4: Calculate $CQ_t = Q_t \times G_t$
5: **end procedure**
6: **procedure** LOGICAL SKILL GRAPH CONSTRUCTION
7: Calculate edit distance $dw(C_t, C_{t+1})$
8: Construct logical skill graph using dw and neighboring nodes
9: **end procedure**
10: **procedure** GRAMMAR SKILL GRAPH CONSTRUCTION
11: Calculate similarity S_{ij} between problems
12: Construct grammatical skill graph using S_{ij} and neighboring nodes
13: **end procedure**
14: **procedure** COLLABORATIVE PREDICTION
15: Integrate logical and grammatical skills: $h_t = f(LSTM, h'_{gt}, h'_{lt}, C_t, G_t, R_t)$
16: Generate P_{t+1}
17: **end procedure**

4.1. Source code information extraction

The source code submitted by students contains rich information, which can essentially be used to assess students' coding abilities (Brown, Kölling, Kyfonidis, & Weill-Tessier, 2022). In order to evaluate students' coding levels as accurately as possible, we assess their code from two dimensions: code quality and code grammar. The specific evaluation methods are shown in Eqs. (1) and (2). To fully

utilize the interaction effects between $G(c_t)$ (grammar score) and $Q(c_t)$ (quality score), we calculate CQ_t in the following manner:

$$CQ_t = G \cdot Q, \quad (3)$$

where G and Q are the embedding corresponding to the grammar score and quality score, respectively.

4.2. Reweighted logical skills evolution graph

Students' demonstration of logical skills during programming exercises is crucial for evaluating their programming abilities (Marjan, Uddin, & Ibn Afjal, 2023). In online assessment systems, the system provides timely feedback on the students' submitted code, including its correctness and performance. To find the optimal solution, students often need to attempt and adjust multiple times, leading to multiple submissions of the same problem. Inspired by previous work (Zhang & Shasha, 1989) on edit distance, we can determine the changes made by students between consecutive submissions by observing the modifications in their code, thereby obtaining the distance dw between two consecutive submissions. Let c_t and c_{t+1} be two consecutive submissions for the same problem, where the node set of c_t is $N_{c_t} \in \{n_1, n_2, \dots, n_i\}$, and the node set of c_{t+1} is $N_{c_{t+1}} \in \{m_1, m_2, \dots, m_j\}$. We define a matrix $\mathbb{D}$, where $D_{i,j}$ represents the edit distance between the subtree rooted at n_i in c_t and the subtree rooted at m_j in c_{t+1} . This calculation can be performed using the following recursive formula:

$$D_{i,j} = \begin{cases} j & \text{if } i = 0, \forall j \in \{0, 1, 2, \dots, m\}, \\ i & \text{if } j = 0, \forall i \in \{0, 1, 2, \dots, n\}, \\ \min(D_{i-1,j} + \delta(n_i, \emptyset), D_{i,j-1} + \delta(\emptyset, m_j), D_{i-1,j-1} + \delta(n_i, m_j)), & \end{cases} \quad (4)$$

$$dw(c_t, c_{t+1}) = \sum_{i=0}^n \sum_{j=0}^m D_{i,j},$$

where $\delta(n_i, \emptyset)$ is the cost of deleting node n_i , $\delta(\emptyset, m_j)$ is the cost of inserting node m_j , and $\delta(n_i, m_j)$ is the cost of replacing node n_i with m_j , and $\min(\cdot)$ is the minimum value function.

To represent the evolution of students' logical skills during the programming process, we measure these skills by observing changes between different submissions of the same problem. These changes manifest as differences in edit distances, source code modifications, and variations in quality and grammar. After obtaining these changes between the codes, we need to construct a sequence graph based on

these changes and the evolutionary path of the code. This sequence graph can be referred to as the reweighted logical skills evolution graph. It takes the students' submitted code c and the code quality grammar score CQ as input data V , and constructs the graph with directed edges E representing the order of code evolution. The specific calculation formula is as follows:

$$\begin{aligned} l_{x_{t-1}} &= W_1(GQ_{t-1} \oplus g(c_{t-1})) + b_1, \\ h'_j &= W_2(\text{Relu}(l_{x_{t-1}})) + b_2, \\ h'_{it} &= \sigma \left(\sum_{j \in \tilde{N}(t)} f(dw(c_j, c_t)) \frac{1}{\sqrt{d_t d_j}} W_3 \text{Relu}(h'_j) \right), \end{aligned} \quad (5)$$

where $g(\cdot)$ is the encoding function using GraphCodeBERT, $\sigma(\cdot)$ is the *softmax* function, $f(\cdot)$ is a normalization function, and $\text{Relu}(\cdot)$ is an activation function. $\oplus$ represents vector concatenation, $W_1 \in \mathbb{R}^{(d_g+768) \times d_g}$, $W_2, W_3 \in \mathbb{R}^{d_g \times d_g}$ are learnable weight parameters, $b_1, b_2 \in \mathbb{R}^{d_g}$ are bias terms, $\tilde{N}(t)$ represents the directed neighbor nodes of code c_t , and d_t and d_j are the degrees of nodes t and j , respectively.

4.3. Grammar skill interaction graph

In programming practice, there is often significant knowledge connectivity between different programming problems (Wang et al., 2017). Similar problems can often be solved using similar solutions. Therefore, accurately assessing the similarity between the problems students have solved, and building a correlation graph of each student's sequence of programming exercises based on this is very important. Specifically, we first encode the global set of programming problems (knowledge concepts) using CodeBERT, then calculate the pairwise similarity using cosine similarity, and form a similarity matrix. The specific calculation is as follows:

$$\begin{aligned} \vec{v}_i &= fc(c_i), \\ S_{ij} &= \frac{\vec{v}_i \cdot \vec{v}_j}{\|\vec{v}_i\| \cdot \|\vec{v}_j\|}, \\ S &= [S_{ij}]_{m \times m}, \end{aligned} \quad (6)$$

where $fc(\cdot)$ is the encoding function of CodeBERT, S_{ij} represents the similarity between two problems, and S is the matrix of programming problem similarities.

Follow this, we will construct the connections between programming problems for each student. We first collect the programming problems involved from the sequence of programming assignments submitted by student S_i . Then, using the similarity matrix S , we select elements that satisfy $S_{ij} \geq t$ (where t is a threshold hyperparameter) as the basis for constructing the knowledge interaction graph between programming problems for that student. By adopting this method, we can not only effectively outline the interconnections between different programming problems for each student but also combine the student's performance on specific programming problems to construct a graph that reflects the interactions of their grammar skills. This graph not only uncovers relationships within the data but also enhances our understanding of students' learning patterns and knowledge construction. Consequently, it allows for a more nuanced grasp of their skill development and mastery of knowledge throughout the programming learning process. The specific calculation method for the grammatical skills interaction graph is detailed below:

$$\begin{aligned} gx_i &= W_{cp}(\vec{v}_i \oplus \mathbf{E}_G[G(c_i)]) + b_{cp}, \\ h_i &= W_4 \text{Relu}(gx_i) + b_4, \\ h'_{gt} &= \sigma \left(\sum_{j \in \tilde{N}(t)} \frac{1}{\sqrt{d_t d_i}} W_5 \text{Relu}(h_j) \right), \end{aligned} \quad (7)$$

where $W_{cp} \in \mathbb{R}^{(d_g+768) \times d_g}$, $W_4, W_5 \in \mathbb{R}^{d_g \times d_g}$ are learnable weight parameters, $b_{cp}, b_4 \in \mathbb{R}^{d_g}$ are bias terms, $\tilde{N}(t)$ represents the directed neighbor nodes of problem cp_t , and d_t and d_i are the degrees of nodes t and i respectively.

4.4. Collaborative output layer

Through the reweighted logical skill evolution graph, we capture the dynamic changes in students' logical skills during multiple consecutive submissions on the same problem, represented as h'_{it} . Simultaneously, using the grammatical skills interaction graph, we depict the representation of knowledge interaction between different programming problems as h'_{gt} . A student's programming ability is a combination of logical and grammatical skills; thus, adequately modeling these skills is key to comprehensively assessing a student's programming level.

Specifically, we combine the learned skill features with the code snippets submitted by students, related knowledge concepts, and the students' responses to the problems. We process this information through a sequential learning process to obtain the student's knowledge states at time step t , denoted as h_t . Then, to more accurately reflect how students apply the learned knowledge in practice, we merge the KC at time step $t+1$, kc_{t+1} , with the student's knowledge state representation to make predictions at that moment. The formula is as follows:

$$\begin{aligned} h_t &= LST M(\alpha h'_{gt} \oplus (1-\alpha)h'_{it} \oplus kc_t \oplus g(c_t) \oplus r_t), \\ pre_t &= h_t \oplus kc_{t+1}, \\ pre_{t+1} &= \sigma(w_t(p_t + b_t)), \end{aligned} \quad (8)$$

where $w_t \in \mathbb{R}^{1 \times (d+d)}$ is a learnable weight parameter, and $b_t \in \mathbb{R}^1$ is a bias term.

4.5. Objective function

To train all parameters within the LGS-KT model, we employ the cross-entropy logarithmic loss function as the objective function, which is based on the discrepancy between the model's predicted value pre and the actual result r of whether the student's submitted code is accepted by the system.

$$\mathcal{L}(\beta) = - \sum_{t=1}^T (r_t \log(pre_t) + (1-r_t) \log(1-pre_t)) + \lambda_\beta \|\beta\|^2, \quad (9)$$

where λ_β represents the set of all parameters in the LGS-KT model, and λ_β refers to the regularization hyperparameter. To minimize the objective function, we utilize the Adam optimizer. The experimental section will further detail the specific configurations of other settings.

5. Experiments

In this section, we will first present the real-world datasets used in our experiments and thoroughly explain the specific details of the training phase, as well as the baseline models selected. Subsequently, this paper will conduct a comparative analysis between the baseline models and our proposed model in predicting students' future learning outcomes. Moreover, to fully demonstrate the advantages of the LGS-KT model, we will explore the following research questions:

- **RQ1** What is the performance of the proposed LGS-KT model in modeling knowledge states?
- **RQ2** Does our proposed LGS-KT model exhibit superior performance in predicting student performance?
- **RQ3** Can the reweighted logical skill evolution graph module effectively represent changes in students' logical skills?
- **RQ4** Can the grammatical skills interaction graph module learn meaningful grammatical skills features?
- **RQ5** How do the various modules in LGS-KT affect the results of KT?
- **RQ6** Does the choice of embedding methods affect the predictive performance of the model?
- **RQ7** How does the LGS-KT model interpret the code submitted by students?

Table 2
Statistics of all datasets.

Datasets	AIZU_C	AIZU_Java	AIZU_Python	AIZU_JavaScript
Statistics				
# of exercise records	159,188	85,908	81,492	9,219
# of students	855	367	369	102
# of programming problems	1,709	1,516	1,493	746
Maximum value of CPU_time	20,000	42,010	42,060	40,000
Maximum value of memory	7,462,732	2,098,380	7,796,236	1,565,056
Average of exercise	186.18	234.08	220.85	90.38

5.1. Datasets

To evaluate the effectiveness of our LGS-KT model, we utilized four real-world datasets for testing. After excluding incomplete data, presents detailed statistics of the meticulously filtered datasets. Specifically, the AIZU dataset was selected from the Project_CodeNet dataset provided by IBM.² This benchmark dataset originates from AIZU.org, a platform highly popular among programming learners. It gathers a vast collection of programming practice records submitted by learners of varying skill levels, ranging from beginners to experienced programmers. The problem sets primarily focus on educational purposes, covering a wide range of topics from basic exercises to complex advanced algorithms. In this study, we particularly focused on several representative programming languages: the procedural programming language *C*, the object-oriented programming language *Java*, the advanced scripting language *Python*, and the web programming language *JavaScript*. Given that online programming data often includes numerous consecutive submissions of the same problem by students, to better assess students' programming skills, our research focuses on those datasets where individual student records exceed 100 entries. For the *JavaScript* dataset, due to the smaller amount of available data, we adjusted the selection criterion for record length to exceed 20 entries.

5.2. Experimental setup

During our experimental process, we excluded entries lacking "problem_id" or "submission_id" and organized the filtered data set accordingly. For all datasets, we segmented the length of the exercises based on the specific length of the data, sequentially splitting entries that exceeded this average length and padding those that fell short with zeros. Specifically, the exercise length for the *AIZU_C* dataset was set to 180, the *AIZU_Java* was set to 230, the *AIZU_Python* was set to 220, and the *AIZU_JavaScript* was set to 90. We initialized all model parameters randomly at the beginning of the training phase. The datasets were partitioned into an 80% training set and a 20% testing set. In terms of hyperparameter tuning, we split a validation set from the training data to optimize model performance and ensure fair comparison. For the LGS-KT model, we standardized the dimensional parameters such as dg, dk, dr, and d at a value of 256 to ensure uniformity in the model's architecture. We processed the data in batches of 32 for each training iteration. These experiments were meticulously carried out on a server running 64-bit Ubuntu 20.04.5 LTS, equipped with an Intel Xeon Gold 5218 CPU at 2.30 GHz and a Tesla-V100 GPU with 32 GB of memory, efficiently utilizing the PyTorch framework for our computations.

5.3. Baseline models

To evaluate the performance of the LGS-KT model, we compare it with 12 different established baseline models. We divide it into two main categories: one is knowledge tracking based on neural-learned

embedding representations; the other is knowledge tracking that integrates pre-trained language models with rich semantic information. With careful replication and precise fine-tuning, we ensure that these models operate at their peak performance, maintaining a fair comparison standard. Our comparative study relies on two key performance indicators: the Area Under the Curve (AUC) and Accuracy (ACC), to provide a comprehensive assessment of efficacy.

5.3.1. Neural-learned embedding

- **LR** (Cox, 1958) is used for knowledge tracing by analyzing students' correct and incorrect answers on exercises to estimate the probability that they have mastered the knowledge. The model takes the result of each exercise as input and outputs a probability indicating whether the student has mastered the corresponding knowledge point, thereby dynamically updating the estimate of the student's knowledge state.
- **BKT** (Corbett & Anderson, 1994) assumes that the learning process follows a Markov process and uses a Hidden Markov Model to represent the student's knowledge state. Specifically, BKT treats the student's knowledge state as a hidden variable and infers changes in this state by observing the student's performance on exercises.
- **DKT** (Piech et al., 2015), as the trailblazing model in the domain of KT that incorporates deep learning techniques, utilizes Recurrent Neural Networks (RNN) or Long Short-Term Memory (LSTM) units to deduce the level of students' understanding and proficiency in various subjects.
- **DKVMN** (Zhang et al., 2017) enhances traditional memory-augmented neural networks by introducing a structured memory system based on key-value pairs. The key component serves to identify relevant memory slots, and the value component is used to store and update information. This method allows for a more dynamic and detailed representation of student knowledge states.
- **GKT** (Nakagawa et al., 2019) pioneers the integration of Graph Neural Networks (GNN) into KT by converting the structure of knowledge into graph form and redefining the task of KT as a node-level classification problem over time within the GNN framework.
- **DIMKT** (Shen, Huang, et al., 2022) incorporates difficulty levels into problem representations, using a layered neural network to link a student's knowledge states with the varying difficulty of problems over time. The model uses two inputs: author-described embeddings and pre-trained code language model embeddings.
- **QIKT** (Chen et al., 2023) adeptly tracks the nuanced changes in a student's understanding by securing cognitive representations tailored to individual questions, utilizing a knowledge acquisition and problem-solving module focused on those questions. The model uses two inputs: author-described embeddings and pre-trained code language model embeddings.
- **TCKT** (Huang et al., 2024) incorporates a causal self-attention mechanism inspired by front-door adjustment theory. This enhancement strengthens the representation of interactions and reduces prediction errors caused by inherent biases in the dataset. The model utilizes two types of inputs: author-described embeddings and pre-trained code language model embeddings.

² <https://dax-cdn.cdn.appdomain.cloud/dax-project-codenet/1.0.0/readme.html>

Table 3

Performance comparison (%) of different methods on various datasets. Highlight the best result in bold, the second best result with underscore, a $\uparrow$ indicates improvement relative to the model without GraphCodeBERT, and a $\downarrow$ indicates a decrease relative to the model without GraphCodeBERT. GCB refers to using GraphCodeBERT encoding. In the outcomes, * denotes a p -value less than 0.05 in t-test.

Category	Methods	AIZU_C		AIZU_Java		AIZU_Python		AIZU_JavaScript	
		AUC	ACC	AUC	ACC	AUC	ACC	AUC	ACC
Neural-learned Embeddings	LR (Cox, 1958)	55.12	51.25	52.34	50.55	52.46	51.87	52.23	51.53
	BKT (Corbett & Anderson, 1994)	69.46	67.32	55.46	61.45	62.12	61.46	62.67	53.56
	DKT (Piech et al., 2015)	74.27	71.01	73.07	67.82	73.15	71.00	73.55	66.85
	DKVMN (Zhang et al., 2017)	74.88	71.25	68.37	65.52	73.27	71.62	72.59	69.04
	GKT (Nakagawa et al., 2019)	75.22	72.14	73.98	68.41	74.52	71.25	74.21	68.94
	DIMKT (Shen, Huang, et al., 2022)	79.08	73.15	78.26	70.11	85.21	76.24	76.25	67.08
	QIKT (Chen et al., 2023)	75.89	73.20	73.74	68.67	73.56	72.35	73.29	68.47
	TCKT (Huang et al., 2024)	76.02	72.31	74.51	69.77	73.99	73.21	72.28	67.27
Pretrained Model-based Embeddings	CodeBERT (Feng et al., 2020)	74.58	71.84	73.55	67.94	73.71	71.45	73.72	67.10
	GraphCodeBERT (Guo et al., 2020)	74.99	72.11	73.42	68.26	73.84	72.03	74.01	67.97
	Code-DKT (Shi et al., 2022)	74.55	71.20	73.18	68.77	73.22	69.25	73.67	66.87
	PDKT (Zhu et al., 2022)	79.16	72.55	76.15	70.16	85.24	74.26	76.48	68.24
	EKT (Liu et al., 2019)	78.42	71.57	77.59	71.06	80.47	75.11	75.56	67.99
	PST (Li et al., 2022)	76.20	72.58	<u>78.28</u>	<u>71.28</u>	<u>85.29</u>	75.24	78.51	68.54
	DIMKT+GCB (Shen, Huang, et al., 2022)	79.99 $\uparrow$	73.01 $\downarrow$	78.34 $\uparrow$	71.01 $\uparrow$	85.14 $\downarrow$	76.98 $\uparrow$	76.74 $\uparrow$	67.52 $\uparrow$
	QIKT+GCB (Chen et al., 2023)	76.25 $\uparrow$	<u>73.31</u> $\uparrow$	73.70 $\downarrow$	69.71 $\uparrow$	76.69 $\downarrow$	74.62 $\uparrow$	74.18 $\uparrow$	68.51 $\uparrow$
	TCKT+GCB (Huang et al., 2024)	75.95 $\downarrow$	73.02 $\uparrow$	75.37 $\uparrow$	70.26 $\uparrow$	74.17 $\uparrow$	74.00 $\uparrow$	71.84 $\downarrow$	67.02 $\downarrow$
Ours	LGS-KT	81.37 *	74.92 *	81.26 *	72.84 *	87.25 *	78.55 *	<u>78.46</u> *	69.48 *

5.3.2. Pretrained model-based embedding

- **CodeBERT** (Feng et al., 2020) as a tool for learning code representations, aligns with the requirements for tracking the progression of programming skills. Utilizing CodeBERT to encode source code, we model the comprehensive learning trajectory of learners by integrating code embeddings, exercise embeddings, and feedback embeddings within an LSTM network architecture.
- **GraphCodeBERT** (Guo et al., 2020) encodes source code, enhancing the foundational insights provided by CodeBERT with an additional layer of structural code information. By integrating code embeddings, exercise embeddings, and feedback embeddings, we utilize an LSTM network architecture to intricately map out the detailed learning trajectories of learners.
- **Code-DKT** (Shi et al., 2022) is a code-based deep KT system that utilizes attention mechanisms to automatically extract and select domain-specific code features to enhance the DKT model. It employs code2vec (Alon et al., 2019) to embed the students' submitted code.
- **PDKT** (Zhu et al., 2022) establishes a bipartite graph to facilitate the embedding of programming problems and introduces an improved pre-trained model, PLCodeBERT, specifically tailored for code embeddings. Additionally, it employs a dual-sequence RNN framework with an attention mechanism featuring exponential decay to effectively integrate features.
- **EKT** (Liu et al., 2019) evaluates the importance of every exercise in relation to all knowledge concepts, taking into account the semantic nuances of the exercises as its input. We utilize GraphCodeBERT (Guo et al., 2020) as the input embedding layer for this model.
- **PST** (Li et al., 2022) captures the characteristics of learners' code when solving problems by designing the Code Information Graph (CIG), and to measure the changes between consecutive submissions, it further developed the Code Tracking Graph (CTG).

5.4. Performance in modeling knowledge states (RQ1)

Accurately modeling students' knowledge states is crucial for KT tasks. In programming exercises, a problem typically contains multiple code submissions by a student, making it essential to capture unique knowledge states within programming tasks through the LGS-KT model to assess students' programming abilities effectively. As shown in

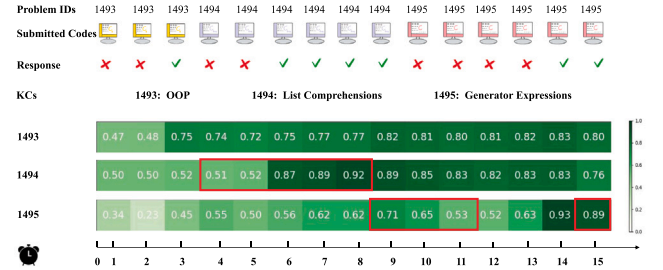


Fig. 3. The LGS-KT model tracks the evolution of a student's knowledge state over time as he/she solve three programming problems.

Fig. 3, the LGS-KT model illustrates changes in students' knowledge states across three programming problems.

Several important phenomena can be observed from the figure. Firstly, the model shows that students' knowledge states undergo significant changes as they deepen their practice. Secondly, for the same problem, as the code iterates, students' knowledge states also change, indicating that our proposed iterative skill assessment significantly impacts knowledge states. For example, in dealing with problem 1494, from time step 4 to 8, as the students' submissions pass system checks, their knowledge states noticeably improve. Furthermore, even when the code passes the checks, the knowledge state continues to change with iterations rather than remaining static, further proving the importance of iteration. Thirdly, it is observed that students' knowledge states are also influenced by grammatical skill interactions. For example, before handling problem 1495 at time step 9, due to the strong association between problems 1494 and 1495, the change in the knowledge state of 1494 also impacts the state of 1495. Lastly, we find that for problem 1495, although the student correctly answered the problem and improved the code at time steps 14 and 15, the knowledge state actually declined. Analysis of the submitted code revealed that the quality of the code at time step 15 had declined compared to time step 14, with increased memory and CPU consumption, indicating that our model can capture such quality changes and has good interpretability. This experiment validates the positive significance of our proposed skill interaction and iteration in modeling students' knowledge states.

5.5. Future performance prediction (RQ2)

The LGS-KT model aims to build a realistic model reflecting students' programming skill states by analyzing their historical programming activities. The extended goal of the model is to provide targeted incentives and interventions for students at different programming skill levels. Our intention is not merely to predict students' future programming performance but to more accurately assess their programming skill states by improving the accuracy of predictions about their future performance. To verify the effectiveness of the LGS-KT model in predicting students' future programming abilities, we conducted a comparative analysis with existing baseline models. In the programming tests conducted for each student, the answers to all problems except the first one were used to predict programming skills at the corresponding time points. This comparative analysis allows us to more comprehensively understand the application potential and advantages of the LGS-KT model in the field of computer education and to provide more effective personalized programming educational support based on this.

To ensure the reliability of the evaluation results, we used two metrics in all experiments: Area Under the Curve (AUC) and Accuracy (ACC). details the experimental results and compares the statistical significance differences between our model and the best baseline model.

From the analysis of , we made several key findings. Firstly, except for slightly lower AUC performance on the AIZU_JavaScript dataset compared to the PST model, the LGS-KT model significantly outperformed the baseline methods in all other metrics. This finding highlights the importance of considering logical and grammatical skills in assessing students' learning outcomes during their programming learning process. Secondly, when the model was pretrained on the code submitted by students, performance improved on most datasets. For example, the DIMKT model showed improvement in six out of eight metrics across four datasets, with only minor declines in the other two metrics. This indicates that a deeper representation of the code submitted by students can more accurately assess their programming abilities. Additionally, we found that KT models specifically designed for programming problems generally outperformed models that performed well on other datasets, such as Code-DKT, PDKT, PST, and the performance of machine learning-based LR and BKT models is significantly lower primarily because they cannot effectively capture and handle complex learning patterns and nonlinear relationships as deep learning models can. Finally, compared to the PST model, which constructs code information graphs and code transformation graphs to analyze the source code submitted by students, the LGS-KT model demonstrated superior performance. This result motivates us to further investigate the differences and connections between multiple submissions of the same programming problem by students, providing valuable experiences for future research and practice. These findings offer new research perspectives in the field of knowledge tracing in computer education and are instructive for future research and practice.

5.6. Logical skills evolution (RQ3)

The ability of the LGS-KT model to effectively represent changes in students' logical skills has a significant impact on the evaluation of their programming skills. We analyzed ten evolving programming codes submitted by two randomly selected students from the AIZU_Python and AIZU_C datasets. Fig. 4 shows the results from the reweighted logical skill evolution graph. From this figure, we can draw several important findings. Initially, as observed in Figure a, the students' codes did not pass the system checks in the early submissions, and their GQ scores were generally low, indicating a lower level of logical skills, which aligns with our understanding. As the students began to submit codes that correctly passed the system checks, their logical skills significantly improved. However, during the sixth and seventh code

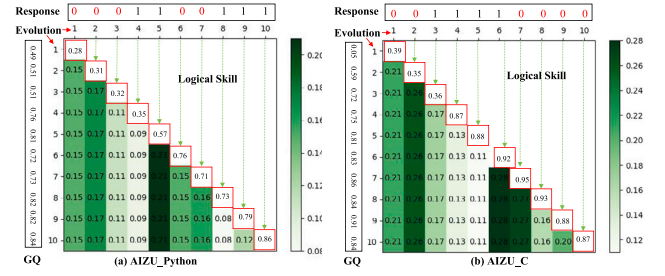


Fig. 4. This presentation showcases 10 evolving programming codes submitted by two students randomly selected from the AIZU_Python and AIZU_C datasets. The "Response" indicates whether the students' submitted codes passed the evaluation, where 1 represents a pass and 0 represents a fail. The 'GQ' value is a standardized score calculated from the quality and grammar scores of student code submissions.

evolution stages, the students made changes to the code that led to errors, and the GQ scores decreased, but there was no significant fluctuation in logical skills. This is mainly because the reweighted logical skill evolution graph integrates learning from all previous performances, not just focusing on the most recent ones.

Observing Figure b, we also noticed a similar pattern, where the students' logical skills significantly changed as their codes passed the system checks. However, a difference was observed during the seventh to tenth code evolution stages, where the students continuously made errors, yet the GQ of the code did not decline, and the students' logical skills remained at a relatively high level. This result aligns well with cognitive patterns of the students; just because their codes did not pass the system checks does not mean a decline in their abilities. It could be that the students were temporarily unable to pass the checks as they pursued better performance, an issue with the programming system setup, not indicative of a decline in their abilities. This experiment demonstrates that the reweighted logical skill evolution graph module can effectively and interpretably evaluate changes in students' logical skills, further providing strong support for assessing students' programming skills.

5.7. Grammatical skills interaction (RQ4)

The LGS-KT model can effectively capture the associations between students' grammatical knowledge, demonstrating excellent interpretability and accuracy. Therefore, training the model to capture the interactive features between students' grammatical knowledge is of significant importance. The grammatical knowledge that students possess plays a crucial role in their ability to write correct code. By training the model to learn the grammatical connections between problems and further interacting these features, an accurate assessment of students' programming abilities can be effectively enhanced.

Fig. 5 shows the grammatical interactions of a student tracked using the LGS-KT model while completing a series of exercises. Several key points are observed in Fig. 5. Firstly, Figure a shows that our LGS-KT model can identify the correlations between the student's grammatical knowledge, for example, exercise 26 shows clear grammatical similarities with exercises 6, 14, 40, and 46. Secondly, we note that the model can essentially learn the associations between grammatical knowledge. For instance, after model training, the representations of exercise 26 are very close to those of exercises 14, 40, and 46; observations from Figure b also indicate that manually annotated grammatical knowledge shows strong associations between exercise 26 and exercises 14, 40, and 46, which are consistent with the grammatical connections learned by the model. Thirdly, from Figure a, we can observe that even exercises without direct connections might end up having very similar representations, such as exercises 26 and 10. This may be because the model is capable not only of learning features from connected nodes built through prior knowledge but also of mastering other latent

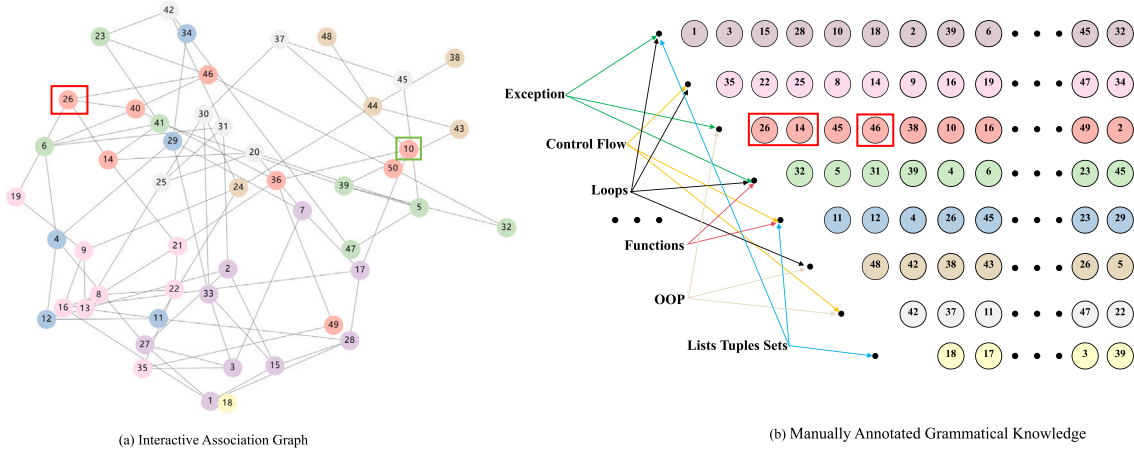


Fig. 5. A student's practice record was randomly selected from the AIZU_Python dataset, and 50 problems were randomly chosen from these records to observe the relationships between them. In figure a, the similarity in node colors represents the closeness of grammatical skill features learned by the LGS-KT model, while the edges indicate similar relationships between them. Figure b shows the grammatical knowledge concepts that might be used by students in solving problems, as manually annotated, where the same color indicates that the solutions to the problems are more similar.

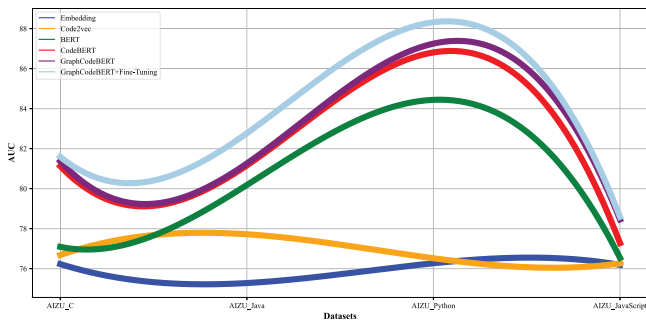


Fig. 6. Comparison of AUC using different embedding methods for code submitted by students in the LGS-KT model.

features between nodes through continuous training. Lastly, we find that although the model can effectively and interpretably represent the existing grammatical knowledge relationships between problems and engage in feature interaction learning, there is still a gap compared to manually annotated grammatical knowledge relations, providing important insights for further utilizing deep learning technologies in programming knowledge tracing research.

5.8. Ablation study (RQ5)

In this section, we will conduct detailed ablation studies on the LGS-KT model to thoroughly examine the specific impact of each module on the overall performance of the model. We will create several variants based on the foundational DKT module, with each variant incorporating a key component. Below are comprehensive descriptions of each variant:

- **DKT** uses LSTM to model the problems and responses during programming exercises.
- **+Pre-trained** module enhances the DKT model by encoding problems through pre-training instead of using traditional embedding methods.
- **+Submitted code** module enhances the processing of the source code information submitted by students and encodes these source codes using GraphCodeBERT.
- **+Logical Skill** module has added tracking of changes in students' logical skills during the continuous evolution of code submission.

- **+Grammatical Skill** module has added tracking of changes in students' grammatical skills during the grammar interaction process of code submission.

Table 4 shows the experimental results for the aforementioned variant models. From the data, it is clear that merely adding the **Pre-trained** module, which involves encoding the problems using CodeBERT without other modifications to the DKT model, does not significantly improve model performance. Further observations reveal that after introducing the **Submitted code** module, there is a noticeable improvement in model performance, strongly supporting the necessity of conducting in-depth analysis of the source code submitted by students. In online programming platforms, a problem may be submitted multiple times with iterative code revisions; the **Logical Skill** module takes into account the changes in students' logical skills during this iterative process. The experimental results show that the inclusion of this module significantly enhances model performance, indicating that considering the changes in logical skills during code iterations is crucial when assessing students' knowledge states. Finally, we observe that the introduction of the **Grammatical Skill** module, completing the full LGS-KT model, has the most significant impact on performance. This demonstrates that considering the grammatical relationships among students when assessing their grammatical skills is vital. These results further emphasize the importance of evaluating students' programming skills in modeling their knowledge states and provide important insights for our research.

5.9. Code embedding strategies (RQ6)

Previous experiments have established the importance of effectively utilizing the code submitted by students. This section explores how different encoding methods influence the model's future predictive performance. We conducted comparative experiments using different embedding techniques on the code submitted by students, and the results are shown in Fig. 6. From the figure, we can see that the performance of the Embedding method is the poorest. The main reason is that this method does not fully utilize the deep information in the code and fails to accurately understand the level of the student's submitted code. In contrast, the GraphCodeBERT encoding method adopted in this paper generally achieves optimal performance. However, we also found that fine-tuning on this encoding method could further enhance performance. Nevertheless, due to the large volume of student code, fine-tuning significantly reduces model efficiency; therefore, we ultimately chose to freeze the embeddings from GraphCodeBERT.

Table 4

Ablation studies of different modules were conducted on the AIZU_C, AIZU_Java, AIZU_Python, and AIZU_JavaScript datasets. The ‘+’ indicates the addition of modules based on the DKT model.

Model	AIZU_C		AIZU_Java		AIZU_Python		AIZU_JavaScript	
	AUC	ACC	AUC	ACC	AUC	ACC	AUC	ACC
DKT	74.27	71.01	73.07	67.82	73.15	71.00	73.55	66.85
+Pre-trained	74.98	72.12	74.37	68.82	82.75	74.21	74.57	67.22
+Summited code	75.01	72.30	74.38	68.91	82.76	74.30	74.62	67.55
+Logical Skill	76.98	73.57	77.21	70.84	84.46	76.37	76.27	68.29
+Grammar Skill $\Rightarrow$ LGS-KT	81.37	74.92	81.26	72.84	87.25	78.55	78.46	69.48

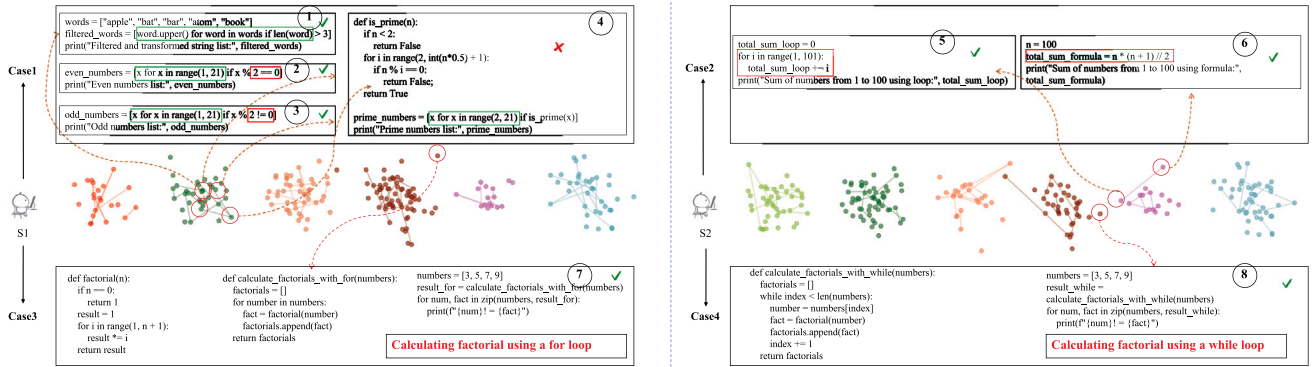


Fig. 7. Randomly select two students from the AIZU_Python dataset and divide them into six clusters in the skill embedding space. Within these clusters, students with very high similarity will be connected by lines to highlight the close relationships between them. Additionally, clusters of students with the same color represent that the LGS-KT model considers their cluster centers to be consistent.

The experiments also show that performance based on BERT models optimized for code (such as BERT, CodeBERT, GraphCodeBERT) generally surpasses other encoding methods. This suggests that considering the semantic features of code is crucial for tasks related to tracking knowledge in programming exercises.

5.10. Case study (RQ7)

The LGS-KT model's ability to accurately capture the programming skills contained in student-submitted code, and the accuracy and interpretability of the information learned by the model, have been demonstrated in the case diagram 7. We analyzed four cases from two randomly selected students from the AIZU_Python dataset. Firstly, by clustering the programming skills of these two students using t-SNE (Hinton & Van Der Maaten, 2008), we were able to effectively differentiate the skill information between students. Secondly, in case 1 corresponding to student S1, we found that four submitted codes were aggregated into the same category, and their relationships were relatively close. For codes 2 and 3, although they were logically similar, differing only in the check for even or odd numbers, they were recognized by the model as the same category due to the use of essentially the same programming grammar. This demonstrates the model's ability to recognize the same grammatical skills in the same student. However, the problem solved by code 1 was different from that of codes 2 and 3, but they were identified as the same category possibly because they used a similar programming grammar format, which also indicates good generalization ability of the model. However, for code 4, although its programming problem belonged to the same category as codes 2 and 3, it was considered to have the same skills because it did not pass the system check, suggesting that further work is needed to more finely distinguish programming skills.

For case 2 corresponding to student S2, codes 5 and 6, from a grammatical skills perspective, these two pieces of code used different syntaxes. Further observation from a logical skill perspective revealed that these two pieces of code actually solved the same problem, only that code 6 was slightly more efficient than code 5, hence the model

considered their programming skills to have a high degree of correlation. This further proves the effectiveness of our proposed Reweighted Logical Skill Evolution Graph.

Additionally, student S1's case 3 and student S2's case 4 were classified as the same type of programming ability. By studying the code submitted by the students, we found that these two cases solved the same problem, just using different loop methods to solve it, and the solutions were essentially the same. This finding gives us great confidence in the generalization ability of our model, in other words, our model can clearly aggregate different students with similar programming abilities and possesses strong grammatical similarity recognition capabilities. This further proves the rationality and high interpretability of our model design.

6. Conclusions and limitations

In this study, we initially collected and processed datasets from four different programming languages on the AIZU platform. Recognizing the distinct nature of programming exercises, we introduced the logical and grammatical skills knowledge tracking model (LGS-KT), which separates students' programming skills into logical and grammatical components for individual modeling. We captured changes in students' logical skills through a reweighted logical skill evolution graph and constructed a global relationship graph to assess grammatical skills. By combining these skill sets, we formed a comprehensive profile of students' programming abilities. Extensive experiments validated the model's effectiveness and interpretability, confirming the importance of considering both skill types. These findings indicate that distinguishing between logical and grammatical skills allows educators to better understand students' programming abilities, enabling more targeted and effective teaching strategies in computer science education.

We acknowledge the contributions of our proposed KT method for programming exercises, especially within the context of programming practice. However, there are some limitations. Firstly, the binary processing of student responses (pass or fail) is simplistic. In reality, testing systems provide feedback on various code states, such as runtime errors and compilation errors. Accounting for these can lead to a

more accurate assessment of students' programming skills. Additionally, this study does not delve into advanced mechanisms for grammar and code quality scoring. The model's performance might be influenced by student backgrounds and teaching environments, affecting its reliability. The choice of datasets is another limitation, as it only includes a few popular programming languages, potentially limiting the model's generalizability. Future research should explore diverse datasets, cross-dataset validation, and contextual factors to enhance the model's robustness.

CRedit authorship contribution statement

Xinjie Sun: Writing – review & editing, Writing – original draft, Visualization, Methodology, Formal analysis. **Qi Liu:** Writing – original draft, Resources, Funding acquisition. **Kai Zhang:** Software, Funding acquisition. **Shuanghong Shen:** Visualization, Software, Methodology. **Yan Zhuang:** Supervision, Software. **Yuxiang Guo:** Visualization, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by Guizhou Provincial Science and Technology Projects (QKHJC[2024]youth012); Guizhou Provincial Higher Education Undergraduate Teaching Content and Curriculum System Reform Project under Grants GZJG2024331 and GZJG2024323; the National Natural Science Foundation of China under Grants 62337001, 62406303, 62261034, and 62177031; Guizhou Province First-Class Undergraduate Course under Grant GZSy1kc202229; Anhui Provincial Natural Science Foundation under Grant 2308085QF229; Anhui Science and Technology Innovation Plan under Grant 202423k09020010; the Fundamental Research Funds for the Central Universities under Grant WK2150110034; Humanities and Social Sciences Research Project of Guizhou Provincial Universities under Grant 2024RW171; and Guizhou Provincial Philosophy and Social Science Research Special Fund (Research on the Spirit of the National Education Conference).

Data availability

Data will be made available on request.

References

- Abdelrahman, Ghodai, & Wang, Qing (2019). Knowledge tracing with sequential key-value memory networks. In *Proceedings of the 42nd international ACM SIGIR conference on research and development in information retrieval* (pp. 175–184).
- Alon, Uri, Zilberstein, Meital, Levy, Omer, & Yahav, Eran (2019). Code2vec: Learning distributed representations of code. *Proceedings of the ACM on Programming Languages*, 3(POPL), 1–29.
- Ashurova, M. M., & Ashurov, M. U. (2023). The role and significance of the concepts of hard skill and soft skill in teaching it and programming languages. *Journal of Pedagogical Inventions and Practices*, 18, 68–70.
- Brown, Neil C. C., Kölling, Michael, Kyfionidis, Charalampos, & Weill-Tessier, Pierre (2022). Getting started with source code analysis for programming education research. In *Proceedings of the 53rd ACM technical symposium on computer science education* v. 2 (pp. 1195–1195).
- Castro-Wunsch, Karo, Ahadi, Alireza, & Petersen, Andrew (2017). Evaluating neural networks as a method for identifying students in need of assistance. In *Proceedings of the 2017 ACM SIGCSE technical symposium on computer science education* (pp. 111–116).
- Chen, Jiahao, Liu, Zitao, Huang, Shuyan, Liu, Qiongqiong, & Luo, Weiqi (2023). Improving interpretability of deep sequential knowledge tracing models with question-centric cognitive representations. *arXiv preprint arXiv:2302.06885*.
- Cheung, Lap Pong, & Yang, Haiqin (2017). Heterogeneous features integration in deep knowledge tracing. In *Neural information processing: 24th international conference, ICONIP 2017, Guangzhou, China, November 14–18, 2017, proceedings, part II 24* (pp. 653–662). Springer.
- Choi, Wan-Chong, Lam, Chan-Tong, & Mendes, António José (2023). A systematic literature review on performance prediction in learning programming using educational data mining. In *2023 IEEE frontiers in education conference* (pp. 1–9). IEEE.
- Corbett, Albert T., & Anderson, John R. (1994). Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4, 253–278.
- Cox, David R. (1958). The regression analysis of binary sequences. *Journal of the Royal Statistical Society. Series B. Statistical Methodology*, 20(2), 215–232.
- Devlin, Jacob, Chang, Ming-Wei, Lee, Kenton, & Toutanova, Kristina (2018). Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*.
- Feng, Zhangyin, Guo, Daya, Tang, Duyu, Duan, Nan, Feng, Xiaocheng, Gong, Ming, et al. (2020). Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*.
- Gao, Ge, Marwan, Samiha, & Price, Thomas W. (2021). Early performance prediction using interpretable patterns in programming process data. In *Proceedings of the 52nd ACM technical symposium on computer science education* (pp. 342–348).
- Ghosh, Aritra, Heffernan, Neil, & Lan, Andrew S. (2020). Context-aware attentive knowledge tracing. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 2330–2339).
- Guo, Daya, Ren, Shuo, Lu, Shuai, Feng, Zhangyin, Tang, Duyu, Liu, Shujie, et al. (2020). Graphcodebert: Pre-training code representations with data flow. *arXiv preprint arXiv:2009.08366*.
- Hermans, Felienne (2020). Hedy: a gradual language for programming education. In *Proceedings of the 2020 ACM conference on international computing education research* (pp. 259–270).
- Hinton, Geoffrey E., & Salakhutdinov, Ruslan R. (2006). Reducing the dimensionality of data with neural networks. *Science*, 313(5786), 504–507.
- Hinton, G., & Van Der Maaten, L. (2008). Visualizing data using t-sne *Journal of machine learning research*, 9, 2579–2605.
- Huang, Yun, Brusilovsky, Peter, Guerra, Julio, Koedinger, Kenneth, & Schunn, Christian (2023). Supporting skill integration in an intelligent tutoring system for code tracing. *Journal of Computer Assisted Learning*, 39(2), 477–500.
- Huang, Changqin, Wei, Hangjie, Huang, Qionghao, Jiang, Fan, Han, Zhongmei, & Huang, Xiaodi (2024). Learning consistent representations with temporal and causal enhancement for knowledge tracing. *Expert Systems with Applications*, Article 123128.
- Iyer, Srinivasan, Konstantinos, Ioannis, Cheung, Alvin, & Zettlemoyer, Luke (2016). Summarizing source code using a neural attention model. In *54th annual meeting of the association for computational linguistics 2016* (pp. 2073–2083). Association for Computational Linguistics.
- Kasurinen, Jussi, & Nikula, Uolevi (2009). Estimating programming knowledge with Bayesian knowledge tracing. *ACM SIGCSE Bulletin*, 41(3), 313–317.
- Keuning, Hieke, Jeuring, Johan, & Heeren, Bastiaan (2023). A systematic mapping study of code quality in education. In *Proceedings of the 2023 conference on innovation and technology in computer science education v. 1* (pp. 5–11).
- Li, Ruixin, Yin, Yu, Dai, Le, Shen, Shuanghong, Lin, Xin, Su, Yu, et al. (2022). PST: measuring skill proficiency in programming exercise process via programming skill tracing. In *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval* (pp. 2601–2606).
- Lindberg, Renny S. N., Laine, Teemu H., & Haaranen, Lassi (2019). Gamifying programming education in K-12: A review of programming curricula in seven countries and programming games. *British Journal of Educational Technology*, 50(4), 1979–1995.
- Liu, Qi, Huang, Zhenya, Yin, Yu, Chen, Enhong, Xiong, Hui, Su, Yu, et al. (2019). Ekt: Exercise-aware knowledge tracing for student performance prediction. *IEEE Transactions on Knowledge and Data Engineering*, 33(1), 100–115.
- Liu, Naiming, Wang, Zichao, Baraniuk, Richard, & Lan, Andrew (2022). Open-ended knowledge tracing for computer science education. In *Proceedings of the 2022 conference on empirical methods in natural language processing*.
- Marjan, Md Abu, Uddin, Md Palash, & Ibn Afjal, Masud (2023). An educational data mining system for predicting and enhancing tertiary students' programming skill. *The Computer Journal*, 66(5), 1083–1101.
- Messer, Marcus, Brown, Neil C. C., Kölling, Michael, & Shi, Miaoqing (2024). Automated grading and feedback tools for programming education: A systematic review. *ACM Transactions on Computing Education*, 24(1), 1–43.
- Minn, Sein, Yu, Yi, Desmarais, Michel C., Zhu, Feida, & Vie, Jill-Jenn (2018). Deep knowledge tracing and dynamic student classification for knowledge tracing. In *2018 IEEE international conference on data mining* (pp. 1182–1187). IEEE.
- Nakagawa, Hiromi, Iwasawa, Yusuke, & Matsuo, Yutaka (2019). Graph-based knowledge tracing: modeling student proficiency using graph neural network. In *IEEE/WIC/ACM international conference on web intelligence* (pp. 156–163).
- Pandey, Shalini, & Karypis, George (2019). A self-attentive model for knowledge tracing. *arXiv preprint arXiv:1907.06837*.
- Piech, Chris, Bassen, Jonathan, Huang, Jonathan, Ganguli, Surya, Sahami, Mehran, Guibas, Leonidas J., et al. (2015). Deep knowledge tracing. *Advances in Neural Information Processing Systems*, 28.

- Qureshi, Muhammad Aasim, Asif, Muhammad, & Anwar, Saira (2023). NewBee: Context-free grammar (CFG) of a new programming language for novice programmers. *Intelligent Automation and Soft Computing*, 37(1), 439–453.
- Rivers, Kelly, Harpstead, Erik, & Koedinger, Kenneth R. (2016). Learning curve analysis for programming: Which concepts do students struggle with? In *ICER: vol. 16*, (pp. 143–151). ACM.
- Shen, Shuanghong, Chen, Enhong, Liu, Qi, Huang, Zhenya, Huang, Wei, Yin, Yu, et al. (2022). Monitoring student progress for learning process-consistent knowledge tracing. *IEEE Transactions on Knowledge and Data Engineering*.
- Shen, Shuanghong, Huang, Zhenya, Liu, Qi, Su, Yu, Wang, Shijin, & Chen, Enhong (2022). Assessing student's dynamic knowledge state by exploring the question difficulty effect. In *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval* (pp. 427–437).
- Shen, Shuanghong, Liu, Qi, Huang, Zhenya, Zheng, Yonghe, Yin, Minghao, Wang, Minjuan, et al. (2024). A survey of knowledge tracing: Models, variants, and applications. *IEEE Transactions on Learning Technologies*.
- Shi, Yang, Chi, Min, Barnes, Tiffany, & Price, Thomas (2022). Code-DKT: A code-based knowledge tracing model for programming tasks. arXiv preprint [arXiv:2206.03545](https://arxiv.org/abs/2206.03545).
- Shi, Yang, Schmucker, Robin, Chi, Min, Barnes, Tiffany, & Price, Thomas (2023). KC-finder: Automated knowledge component discovery for programming problems. *International Educational Data Mining Society*.
- Sonkar, Shashank, Waters, Andrew E., Lan, Andrew S., Grimaldi, Phillip J., & Baraniuk, Richard G. (2020). Qdkt: Question-centric deep knowledge tracing. arXiv preprint [arXiv:2005.12442](https://arxiv.org/abs/2005.12442).
- Sun, Xinjie, Zhang, Kai, Shen, Shuanghong, Wang, Fei, Guo, Yuxiang, & Liu, Qi (2024). Target hierarchy-guided knowledge tracing: Fine-grained knowledge state modeling. *Expert Systems with Applications*, Article 123898.
- Sun, Xia, Zhao, Xu, Li, Bo, Ma, Yuan, Sutcliffe, Richard, & Feng, Jun (2021). Dynamic key-value memory networks with rich features for knowledge tracing. *IEEE Transactions on Cybernetics*, 52(8), 8239–8245.
- Svyatkovskiy, Alexey, Deng, Shao Kun, Fu, Shengyu, & Sundaresan, Neel (2020). Intellicode compose: Code generation using transformer. In *Proceedings of the 28th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering* (pp. 1433–1443).
- Wang, Fei, Huang, Zhenya, Liu, Qi, Chen, Enhong, Yin, Yu, Ma, Jianhui, et al. (2023). Dynamic cognitive diagnosis: An educational priors-enhanced deep knowledge tracing perspective. *IEEE Transactions on Learning Technologies*.
- Wang, Fei, Liu, Qi, Chen, Enhong, Huang, Zhenya, Yin, Yu, Wang, Shijin, et al. (2022). NeuralCD: a general framework for cognitive diagnosis. *IEEE Transactions on Knowledge and Data Engineering*.
- Wang, Lisa, Sy, Angela, Liu, Larry, & Piech, Chris (2017). Deep knowledge tracing on programming exercises. In *Proceedings of the fourth (2017) ACM conference on learning@ scale* (pp. 201–204).
- Yang, Yang, Shen, Jian, Qu, Yanru, Liu, Yunfei, Wang, Kerong, Zhu, Yaoming, et al. (2021). GIKT: a graph-based interaction model for knowledge tracing. In *Machine learning and knowledge discovery in databases: European conference, ECML PKDD 2020, Ghent, Belgium, September 14–18, 2020, proceedings, part I* (pp. 299–315). Springer.
- Zaremba, Wojciech, & Sutskever, Ilya (2014). Learning to execute. arXiv preprint [arXiv:1410.4615](https://arxiv.org/abs/1410.4615).
- Zhang, Kaizhong, & Shasha, Dennis (1989). Simple fast algorithms for the editing distance between trees and related problems. *SIAM Journal on Computing*, 18(6), 1245–1262.
- Zhang, Jiani, Shi, Xingjian, King, Irwin, & Yeung, Dit-Yan (2017). Dynamic key-value memory networks for knowledge tracing. In *Proceedings of the 26th international conference on world wide web* (pp. 765–774).
- Zhang, Jian, Wang, Xu, Zhang, Hongyu, Sun, Hailong, Wang, Kaixuan, & Liu, Xudong (2019). A novel neural source code representation based on abstract syntax tree. In *2019 IEEE/ACM 41st international conference on software engineering* (pp. 783–794). IEEE.
- Zhang, Zheng, Wu, Le, Liu, Qi, Liu, Jiayu, Huang, Zhenya, Yin, Yu, et al. (2023). Understanding and improving fairness in cognitive diagnosis. *Science China Information Sciences*.
- Zhu, Renyu, Zhang, Dongxiang, Han, Chengcheng, Gaol, Ming, Lu, Xuesong, Qian, Weining, et al. (2022). Programming knowledge tracing: A comprehensive dataset and a new model. In *2022 IEEE international conference on data mining workshops* (pp. 298–307). IEEE.